

Lab Exercise 1: Triggering Nested Loop Join

QUERY PLAN
text
Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=5.611..10.570 rows=32.00 loops=1)
Buffers: shared read=153
-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=1.840..1.845 rows=1.00 loops=1)
Index Cond: (customer_id = 1)
Index Searches: 1
Buffers: shared read=3
-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=3.757..8.698 rows=32.00 loops=1)
Filter: (customer_id = 1)
Rows Removed by Filter: 16012
Buffers: shared read=150
Planning:
Buffers: shared hit=86 read=4 dirtied=1
Planning Time: 28.094 ms
Execution Time: 12.816 ms

- Outer (Top): The customer table because it is used for WHERE filters
- Inner (Bottom): The rental table. PostgreSQL queries this table (through a foreign key index scan on customer_id) using the values supplied by the outer table.
- The customer table returned 1 row
- **Why does this make Hash Join "too expensive"?:** A hash join requires allocating memory and building a hash table structure for one of the relations. Doing this infrastructure setup for a single row (rows=1) introduces unnecessary CPU and memory overhead
- The inner table's loops count is 1 because the outer table (customer) returned only 1 row.

Lab Exercise 2: Triggering Hash Join

QUERY PLAN

text

Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=7.721..12.214 rows=1000.00 loops=1)

Hash Cond: (f.language_id = l.language_id)

Buffers: shared hit=88 read=1

-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.818..5.054 rows=1000.00 loops=1)

Buffers: shared hit=88

-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=6.430..6.432 rows=6.00 loops=1)

Buckets: 1024 Batches: 1 Memory Usage: 9kB

Buffers: shared read=1

-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=5.972..5.978 rows=6.00 loops=1)

Buffers: shared read=1

Planning:

Buffers: shared hit=162 read=2

Planning Time: 35.449 ms

Execution Time: 14.555 ms

- **Which table is being Hashed (Built)? Which table is being Probed?:**
 - + Hashed (Built): The language table. PostgreSQL chooses the smaller table to build the hash table in memory because it consumes minimal RAM and reduces overhead.
 - + Probed: The film table. PostgreSQL scans the larger film table row-by-row and probes (queries) the hash table created from the language table to find matching rows.
- Memory Usage: 9kB
- When running SET work_mem = '64kB'; and try the previous code again, nothing changes as the language table is too small (below 64kB)

Lab Exercise 3: Triggering Merge Join

QUERY PLAN
text
Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=17.657..23.992 rows=4581.00 loops=1)
Merge Cond: (f.film_id = i.film_id)
Buffers: shared hit=87 read=26, temp read=10 written=12
-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.031..4.188 rows=1000.00 loops=1)
Index Searches: 1
Buffers: shared hit=84 read=1
-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=17.614..18.746 rows=4581.00 loops=1)
Storage: Memory Maximum Storage: 17kB
Buffers: shared hit=3 read=25, temp read=10 written=12
-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=17.326..17.915 rows=4581.00 loops=1)
Sort Key: i.film_id
Sort Method: external merge Disk: 80kB
Buffers: shared hit=3 read=25, temp read=10 written=12
-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=7.667..9.282 rows=4581.00 loops=1)
Buffers: shared read=25
Planning:
Buffers: shared hit=88 read=6
Planning Time: 27.660 ms

- Structure:

- + Sort node applied to the inventory table. PostgreSQL had to perform a full table scan (Seq Scan on inventory i) and then sort the data by the i.film_id key.
- + It also used an Index Scan to get sorted data for "free" for the film table (Index Scan using film_pkey on film f)

QUERY PLAN
text
Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=4.249..6.218 rows=4581.00 loops=1)
Hash Cond: (i.film_id = f.film_id)
Buffers: shared hit=113
-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.060..0.892 rows=4581.00 loops=1)
Buffers: shared hit=25
-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=4.138..4.139 rows=1000.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 59kB
Buffers: shared hit=88
-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.358..2.947 rows=1000.00 loops=1)
Buffers: shared hit=88
Planning:
Buffers: shared hit=6
Planning Time: 20.131 ms
Execution Time: 8.674 ms

- When reset the setting (RESET ALL;) and remove ORDER BY, the execution plan switches back to Hash Join because:
 - + RESET ALL removes your previous restrictions (SET enable_hashjoin = OFF and SET enable_nestloop = OFF) and restores default configurations like work_mem. The PostgreSQL query planner is no longer artificially forced into using a specific strategy and is free to choose the most cost-effective algorithm.
 - + When query included ORDER BY f.film_id, a Merge Join was relatively attractive to the planner because a Merge Join naturally outputs data sorted by the join key, completely avoiding a final sorting step. By removing ORDER BY, the requirement for sorted output was removed.